Compte-Rendu DATA TECHNIQUE du Challenge 48H

1.Sommaire

1) Équipe du pôle DATA……3
2) Contexte du projet……4
3) Exploration du dataset……5
4) Analyse du dataset……6
5) Sélection des données utiles……8
6) Nettoyage des données (date)...……9
7) Nettoyage des données (numérique)……11
8) Export des données nettoyées……12
9) Futur du projet lié au pôle DATA……..12
10) Conclusion……12

1)Équipe du pôle DATA

Jeannot Louis IA DATA B1

Pôle DATA	Rôle
Louis	**Responsable** conception et développement DATA

2) Context du projet et objectif

Dans le cadre du projet, l'objectif était de développer une appli permettant :

- de créer des tournois de babyfoot,
- d'y participer,
- et de consulter l'historique des matchs.

Afin d'alimenter l'appli avec des données pertinentes, un dataset a été récupéré depuis le dépôt GitHub du projet.
Ce dataset contient un historique de matchs de babyfoot enregistrés précédemment.

Le rôle du pôle data était donc de :

- analyser le dataset,
- nettoyer les données,
- sélectionner les informations utiles,
- transformer les données dans un format exploitable par l'appli.

Pour cela, un travail a été mené en collaboration avec l'équipe de développement afin d'identifier les données nécessaires à l'affichage et au fonctionnement de l'appli.

Les informations jugées essentielles pour l'application web sont :

- l'identifiant du match (game_id)
- la date du match (game_date)
- les équipes
- le score des équipes (final_score_red & final_score_blue)
- le nom de l'arbitre (referee)
- l'identifiant des joueurs (player_id)
- le nom des joueurs (player_name)

Ces données permettent de reconstituer l'historique des matchs et d'afficher les résultats de manière claire sur l'appli.

Les objectifs pour le pôle data était:

- Passer en revue tout le dataset
- Repérer les points de blocages et les données pertinentes
- Nettoyer les points repérés précédemment
- Tirer et analyser les données pouvant être pertinente et en accord avec les besoin du pôle dev
- Préparer un fichier exploitable pour appli en flutter et donc une implémentation dans la db
- Tirer plus de donné précise pour ajouter des features comme des stats précise de match, de joueur ou d'arbitre et pouvoir réellement faire de l'analyse de data

Beaucoup de ces objectifs prennent du temps au vue de l'équipe disponible, des compétences dont elle dispose ainsi que le temps dont nous disposons malgré tout une bonne partie de ces objectifs ont pu être atteint mais malheureusement les plus intéressant et les plus compliqués ne pourrait voir le jour qu'avec plus de temps.

3) Exploration du dataset

Afin de travailler sur le dataset, l'environnement suivant a été utilisé :

- **Python**
- **Jupyter Notebook**
- **Pandas**

Pandas est une bibliothèque Python spécialisée dans la manipulation et l'analyse de données.

Elle permet notamment :

- de lire des fichiers CSV
- de manipuler des tableaux de données (DataFrames)
- de nettoyer et transformer des datasets
- d'effectuer des analyses statistiques.

Le dataset a d'abord été chargé dans un DataFrame pandas :

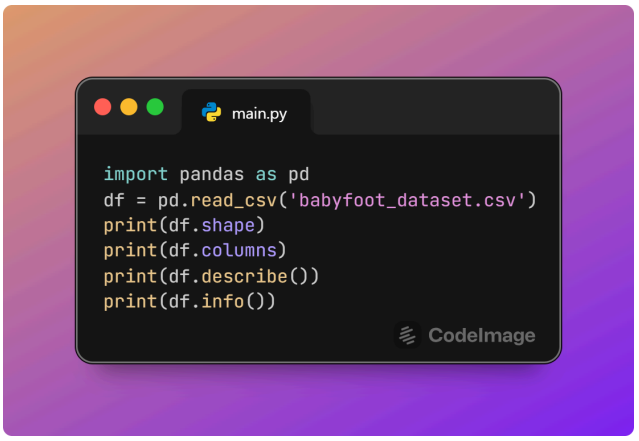
Un DataFrame peut être comparé à un tableau Excel.
En l'affichant avec

ca permet des affichages justement proche du tableau Excel

	game_id	game_date	location	table_id	table_condition	ball_type	music_playing	referee	game_duration	final_score_red	...	possession_time
0	G015295	Feb 06st 2023	Ynov Toulouse	T05	beer stains	NaN	Spotify: Queen - We Will Rock You	Paul Kim	12.45	0	...	9:36
1	G023800	24-03-2023	Cafeteria (1st floor)	T07	worn	NaN	Indie playlist	NaN	8.57	10	...	NaN
2	G023577	2025-01-13	Gym Hall	T26	scratched	NaN	Spotify: Queen - We Will Rock You	Lena Clement	17.2	2	...	5.09min
3	G020644	Nov 11 2025	Salle Polyvalente	T21	worn	mini ball	EDM mix	Isabella Girard	5.18	6	...	NaN
4	G011677	30 Sep 23	Campus - Cafeteria	T26	missing screw	trainer ball	Oldies 80s	yes	6min	3	...	177
5	G019911	2023/07/04	Gym Hall	T26	new	old worn	Lo-fi beats	Youssef Garcia	5min	9	...	NaN
6	G015349	2023/11/16	Ynov Toulouse	T24	scratched	mini ball	Lo-fi beats	no	18min	0	...	6.45min
7	G004849	2025/11/16	Lab 204	T20	out of alignment	3-ball set	Spotify: Queen - We Will Rock You	no	00:08:11	7	...	2.52min
8	G022917	20 Mar 23	Bar Le Foos	T24	new	white hard	Radio 104.5	NaN	14.62	8	...	8.98min
9	G000463	04 Feb 24	Student House	T18	scratched	white hard	EDM mix	NaN	16.11	0	...	7:38

4) Analyse du dataset

La première étape consiste à comprendre la structure globale du dataset.



Taille du dataset

avec : df.shape — (output) — > `(100200, 35)`

Cette fonction permet d'obtenir :

- le nombre de lignes
- le nombre de colonnes

Cela permet de connaître la taille globale du dataset et d'estimer la quantité de données à traiter.

Liste des colonnes

avec : df.columns

```
Index(['game_id', 'game_date', 'location', 'table_id', 'table_condition',
       'ball_type', 'music_playing', 'referee', 'game_duration',
       'final_score_red', 'final_score_blue', 'winner', 'attendance_count',
       'season', 'recorded_by', 'rating_raw', 'player_id', 'player_name',
       'player_canonical_name', 'player_age', 'player_role', 'player_goals',
       'player_own_goals', 'player_assists', 'player_saves', 'possession_time',
       'mood', 'player_comment', 'team_color', 'is_substitute', 'ping_ms',
       'notes', 'duplicate_flag', 'misc', 'created_at'],
      dtype='str')
```

Cette commande permet d'obtenir la liste de toutes les variables présentes dans le dataset.

Chaque colonne représente une information enregistrée sur une partie de babyfoot.

Exemples de variables :

- game_id
- game_date
- location
- table_condition
- final_score_red
- player_name

Cette étape permet également d'identifier les variables réellement utiles pour le projet.

Informations détaillées sur le dataset

avec : df.info()

La fonction info() fournit plusieurs informations importantes :

- le nombre de lignes du dataset
- la liste des colonnes
- le nombre de valeurs non nulles
- le type de données de chaque colonne

Exemple de types de données :

```
<class 'pandas.DataFrame'>
RangeIndex: 100200 entries, 0 to 100199
Data columns (total 35 columns):
 #   Column                Non-Null Count   Dtype
---  ------                --------------   -----
 0   game_id               100200 non-null  str
 1   game_date             100200 non-null  str
 2   location              100200 non-null  str
 3   table_id              100200 non-null  str
 4   table_condition       100200 non-null  str
 5   ball_type             77675 non-null   str
 6   music_playing         90126 non-null   str
 7   referee               40117 non-null   str
 8   game_duration         100200 non-null  str
 9   final_score_red       100200 non-null  str
...
```

```
 22  player_own_goals       100200 non-null  object
 23  player_assists         100200 non-null  object
 24  player_saves           100200 non-null  int64
 25  possession_time        75148 non-null   str
 26  mood                   88970 non-null   str
 27  player_comment         83613 non-null   str
 28  team_color             100200 non-null  str
 29  is_substitute          75024 non-null   str
 30  ping_ms                50024 non-null   str
 31  notes                  66813 non-null   str
 32  duplicate_flag         60333 non-null   str
 33  misc                   40072 non-null   str
 34  created_at             100200 non-null  str
dtypes: int64(1), object(3), str(31)
memory usage: 26.8+ MB
```

Cette étape permet notamment d'identifier :

- les colonnes contenant des valeurs manquantes
- les colonnes dont le type de données n'est pas adapté.

Dans notre cas, l'analyse montre que plusieurs colonnes contiennent un grand nombre de valeurs manquantes, ce qui indique qu'un nettoyage du dataset est nécessaire.

5) Sélection des données utiles

Le dataset initial contient un grand nombre de variables qui ne sont pas nécessaires au fonctionnement de l'appli.

Afin de simplifier le traitement des données, un premier tri des colonnes a été effectué.

Les colonnes conservées sont celles nécessaires pour :

- afficher les matchs
- afficher les scores
- afficher les joueurs
- afficher les arbitres
- afficher les dates

Cette étape permet de :

- réduire la complexité du nettoyage
- simplifier le traitement des données
- améliorer les performances du script.

6) Nettoyage des données (date)

L'une des principales difficultés rencontrées concerne l'hétérogénéité des formats de données.

Par exemple, les dates peuvent apparaître sous plusieurs formats différents (observables sur un des premiers screen montrant le tableau):

- Feb 06st 2023
- 24-03-2023
- 2025-01-13
- 30 Sep 23

Ces formats doivent être convertis dans un format unique afin d'être exploitables par l'application.

Un script de nettoyage a donc été développé pour :

- uniformiser les formats de dates
- corriger certains caractères parasites
- normaliser certaines catégories
- convertir les valeurs numériques lorsque cela est possible.

L'objectif du nettoyage n'est pas de supprimer les données imparfaites, mais de rendre les données suffisamment cohérentes pour être exploitées par l'application.

```python
# Permet de voir les format unique de date
df["game_date"].unique()[:20]
# Permet de d'enlever tout les suffix de date du style st nd rd ou th
df["game_date"] = df["game_date"].astype(str)

df["game_date"] = df["game_date"].apply(
    lambda x: re.sub(r"(\d+)(st|nd|rd|th)", r"\1", x)
)
```

```python
# Commencer la conversion de des dates au même format
df["game_date"] = pd.to_datetime(df["game_date"], dayfirst=True, errors="coerce")
```

Mais les premiers problèmes arrivent certaines dates n'était pas encore au bon format malgré un premier nettoyage.

```python
from dateutil import parser
import re

def clean_date(x):

    if pd.isna(x):
        return None

    x = str(x)

    x = re.sub(r"(\d{1,2})(st|nd|rd|th)", r"\1", x)

    try:
        return parser.parse(x, dayfirst=True)
    except:
        return x
```

j'ai donc utilisé dateutil avec la fonction parse qui montre de meilleur résultat.

7) Nettoyage des données (numérique)

Certaines colonnes contiennent des nombres sous forme de texte.

Par exemple :

game_duration	final_score_red	...	possession_time
12.45	0	...	9:36
8.57	10	...	NaN
17.2	2	...	5.09min
5.18	6	...	NaN
6min	3	...	177
5min	9	...	NaN
18min	0	...	6.45min
00:08:11	7	...	2.52min
14.62	8	...	8.98min
16.11	0	...	7:38

Avec la colonne game_duration our possession_time

main.py

```python
import numpy as np

def clean_numeric(x):

    if pd.isna(x):
        return np.nan

    s = str(x).lower()

    s = s.replace("min","")
    s = s.replace(",",".")
    s = re.sub(r"[^0-9.\-]", "", s)

    try:
        return float(s)
    except:
        return x
```

CodeImage

Avec numpy on a pu rédiger un script permettant de nettoyer tout les caractères alpha numérique.

8) Export des données nettoyées

Une fois les transformations terminées, le dataset est exporté dans un nouveau fichier CSV.

```
main.py
df.to_csv("babyfoot_dataset_clean.csv", index=False)
```

Avec des library importables dans flutter comme csv ou alors simplement en l'utilisant en tant qu'asset dans .yaml il est possible d'utiliser ces nouvelles données exportés.

9) Futur du projet lié au pôle DATA

Dans le futur du pôle DATA serait de continuer à travailler sur les objectifs fixer, une potentiel intégration sur l'appli réaliser en Flutter ainsi que des features complémentaire pourraient voir le jour tels que un tracker de stat sur les joueurs avec leur taux de possession, leur état mental après une victoire ou un défaite, voir le scoring et le taux de victoire en fonction de la musique écouter ou non durant un match. Il en va de même vis-à -vis des équipes mais aussi des arbitres. Voir les tendances concernant les arbitres et les commentaires des joueurs pourrait permettre aussi de vérifier l'arbitrage dans un sens si justement le jugement d'un arbitre suis un pattern autre que simplement les règles.

10) Conclusion

En conclusion, le projet mené par ce pôle fut enrichissant quoi qu'un peu compliqué. Malgré tout, tout le travail effectué laisse de grandes opportunités de développement concret pour le babyfoot, le projet de notre équipe de manière plus globale ainsi que le domaine de la data.